

MITA — Main Tasks for Heuristic Evaluation

Course: Multimodal Interactive Technologies and Applications

This document describes the three interaction tasks on which the evaluating team is expected to run their heuristic evaluation. Each task gives the user goal, the modalities involved, the expected interaction flow and the success criteria. All tasks take place inside the immersive A-Frame botanical garden after the loading screen dismisses and the first panoramic scene (**scene-1**) is visible.

1 Task 1 — Disambiguation of Overlapping Plants

Goal. Ask a question about **one specific plant** in a region where several specimens visually overlap (e.g. the cluster of Lavandula / Nephrolepis / Santolina near the right path in **scene-1**).

Modalities. *Input:* pointer raycasting + speech (STT) + click fallback on the A / B overlay. *Output:* 3D highlight on every candidate, modal overlay with labelled options, and a TTS clarification question.

Flow.

1. User points at the overlapping plants and clicks (or speaks while gazing).
2. The frontend collects all raycaster intersections of class `.plant-model` and stores the candidates in client-side state. If more than one candidate is detected, the ambiguity flow starts immediately on the client.
3. The frontend sorts candidates by depth (closest first), labels them A, B..., and calls `/api/generate-disambiguation` to obtain a spoken clarification prompt (*“I can see two plants — do you mean A, the one in front, or B, behind it?”*).
4. The frontend highlights every candidate, opens the A / B overlay and plays the TTS prompt.
5. User resolves by voice (*“A”, “the one behind”, “lavender”*) or by clicking; the frontend calls `/api/clarify` with the utterance and stored candidate context.
6. The clarify handler resolves the utterance against the candidates and returns a structured response; the card is rendered and the overlay is dismissed.

Success criteria. Highlight, overlay and TTS appear within ~1s of the click; voice resolution succeeds for natural phrasings (*“A”, “the front one”, plant name*); if unresolved, a `clarify_retry` with softer wording is issued and after **two** failed retries the user is guided to the click buttons; no information card is rendered until the ambiguity is resolved.

Heuristics. Match between system and the real world; visibility of system status; user control and freedom (overlay always dismissible); error prevention and recovery (retry + click fallback).

2 Task 2 — Comparing the Current Plant with a Visited One

Goal. After inspecting at least one plant, **compare** a newly selected plant with one seen earlier (*“How does this compare to the lavender I just looked at?”*).

Modalities. *Input:* pointer raycasting to lock onto the new plant + speech (free-form comparison query). *Output:* a dedicated comparison card rendered side-by-side, plus a TTS summary.

Flow.

1. User has previously selected at least one plant; its id and name are stored in the client-side `visitedPlants` list in `src/app/state.js`.
2. User clicks the second plant and asks a comparison-flavoured question (*“compare it with the lavender”, “which one is more drought-tolerant?”*).
3. The frontend calls `/api/parse-intent`, then routes to `/api/compare`, passing the utterance, selected plant and the full `visited` history as request context.
4. The compare handler runs `parseIntent`; if intent is `compare` and a `comparison_target` is identified (by id, alias or recency), it proceeds with both plants.
5. The handler fetches both plants from `plants.json`, optionally filters by the detected `interest_slot` (e.g. `medicinal`, `drought`), and returns a `comparison` payload with both records and a spoken summary.

6. Client renders `renderComparisonCard(plant1, plant2, slot)` and plays the TTS narration.

Success criteria. Implicit references (“*the previous one*”, “*lavender*”) are resolved via the visited list and alias table; the card visually aligns matching attributes (height, toxicity, drought tolerance, family...); comparison only triggers when the user is **not locked** to a single plant via the contextual follow-up path (preventing accidental comparisons in a focused Q&A); a graceful error is returned when no valid second plant can be identified.

Heuristics. Recognition rather than recall (visited plants are remembered for the user); consistency and standards (same card layout as Task 3); aesthetic and minimalist design (only the requested slot is compared, not the full record).

3 Task 3 — Dynamic Information Card Driven by Spoken Intent

Goal. Obtain **targeted information** about a single plant — botanical description, medicinal use, toxicity, drought tolerance, feng-shui symbolism — without navigating menus.

Modalities. *Input:* pointer raycasting to select the plant + speech (STT); follow-up turns include the selected plant id as context so the system stays focused on the displayed plant. *Output:* a context-sensitive card whose layout depends on the detected `interest_slot` (`medical`, `botanical`, `default`), a 3D highlight, and a TTS answer.

Flow.

1. User selects a plant; the frontend records the visit in client-side state and calls `/api/parse-intent` to parse intent and slot.
2. Slot is mapped to a card type via `SLOT_TO_CARD` (`medicinal` → `medical`, `botanical` → `botanical`, others → `default`).
3. If the requested slot is missing (e.g. `feng_shui` for a plant with `feng_shui_info: null`), the server returns a `fallback` payload listing the **slots that do exist**, plus a polite spoken explanation.
4. The frontend renders `renderInfoCard(cardType, data, onFollowUp)` for normal answers, or `renderFallbackCard(msg, onSlotPicked)` for missing data (each button re-issues a request to `/api/generate-info` locked to the same plant).
5. Subsequent utterances are treated as **contextual follow-ups**; dialogue history is passed as request context so `generateContextualAnswer` can produce coherent multi-turn answers.

Success criteria. Card type changes with spoken intent (e.g. “*is it safe for my dog?*” surfaces toxicity content, not the default description); the fallback path never dead-ends — at least one alternative slot is always offered; multi-turn follow-ups remain locked to the selected plant, and switching plants clears the dialogue panel (`clearDialogue()` on `selectedPlantId` change); TTS, card and 3D highlight stay synchronised across turns.

Heuristics. Flexibility and efficiency of use; help users recognise and recover from errors (fallback slots); consistency between voice and visual answers; minimalist design of each card.

Out-of-Scope

Not part of this heuristic evaluation: *scene navigation* between the five panoramic environments via arrow portals (evaluated only as a pre-condition for reaching the plants); *STT/TTS audio quality* itself (inherits the browser-native Web Speech API); *LLM answer factuality* (bounded by the curated `plants.json` database, not a target of the UX evaluation).